과제: MQTT 기반 설비 이상 알람 서비스 구현

Sep 30, 2026 · @김한진

1. 과제 개요

이상 탐지 모델은 이미 제공되며, 학생은 모델의 판정 결과를 MQTT 알람으로 내보내는 알람 서비스(alarm_service.py)만 구현한다. 뼈대 코드의 TODO 1~5를 채우면 되고, 권장 수행 기간은 1주다.

각 TODO는 MQTT 실습의 한 단계와 대응한다.

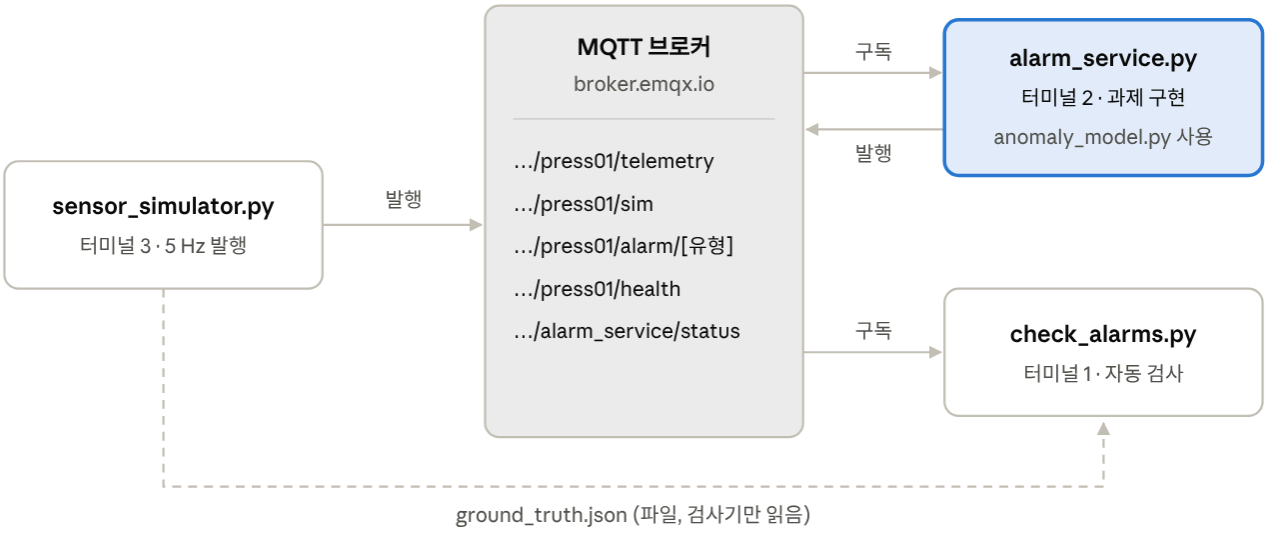
MQTT 실습	과제에서 구현할 내용
Step 1 Pub/Sub	TODO 1: 센서 텔레메트리 구독
(추가) 상태 전이	TODO 2: NORMAL ↔ ALARM 상태 판단
Step 2 토픽 계층	TODO 3: 이상 유형별 알람 토픽(alarm/{유형})으로 발행
Step 3 Retained	TODO 4: 현재 설비 상태(health)를 retain으로 발행
Step 4 LWT	TODO 5: 서비스 상태 ONLINE 발행과 OFFLINE Will 메시지 등록

제출 기한: [제출 기한 입력]

2. 시스템 구성

세 프로세스는 브로커의 토픽으로만 연결된다. 학생이 구현하는 것은 alarm_service.py 하나이며, 이 안에서 제공된 anomaly_model.py를 호출해 판정 결과를 얻는다.

구현 대상은 alarm_service.py, 탐지 모델은 제공된다



과제 시스템 구성 · 프로세스 3개, 토픽 5개

모든 토픽은 [USER_ID]/factory/line1/ 아래에 있다. 검사기는 alarm·health·status 토픽을 구독하여 알람 서비스가 요구사항대로 동작하는지 확인한다. 개발 중에는 watch_alarms.py로 알람 서비스의 발행 결과만 골라 볼 수 있다(5장).

3. 제공 파일

파일 5개 중 학생이 구현하는 것은 alarm_service.py뿐이다. 나머지는 USER_ID만 바꾸고, 시뮬레이터는 실행 모드(RUN_MODE)도 고른다.

파일	역할	수정
sensor_simulator.py	프레스 센서 값 발행. quick(개발용, 10 Hz·900개·약 1분 30초·이상 4건)과 full(제출용, 5 Hz·3,000개·약 10분·이상 12건) 두 모드	USER_ID, RUN_MODE
anomaly_model.py	학습 구간으로 학습하고 샘플마다 이상 여부와 유형을 판정	없음
alarm_service.py	뼈대 코드. TODO 1~5를 채워 완성	과제
watch_alarms.py	개발용 모니터. 센서 값은 숨기고 알람·health·status만 표시하며, 알람이 오면 정답과 바로 비교	USER_ID만
check_alarms.py	제출용 자동 검사기. 8개 항목 통과 여부 판정	USER_ID만

모델은 텔레메트리 메시지를 그대로 받아 판정 결과를 돌려준다. 학습과 호출은 뼈대 코드에 이미 들어 있다.

```
result = model.predict(sample)
# {"anomaly": True, "type": "stuck", "score": 12.5}
# {"anomaly": False, "type": None, "score": 0.8}
```

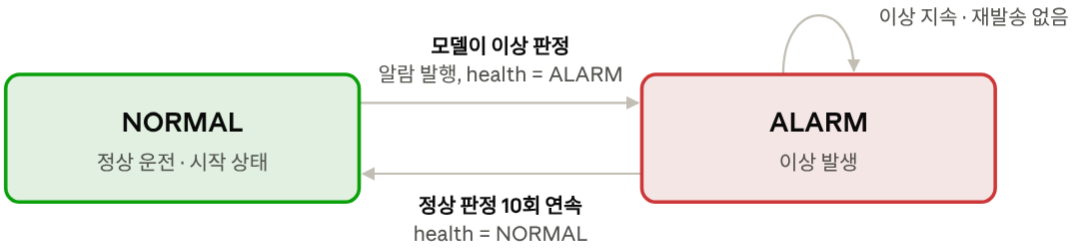
모델이 구분하는 이상 유형은 4가지다.

유형	현장 의미	신호 변화
spike	이물 충격, 순간 과부하	진동 순간 급증
drift	베어링 과열 진행	온도 점진 상승
vib_noise	체결부 풀림	진동 흔들림 증가
stuck	온도 센서 고착	온도 값 고정

4. 구현 내용

알람 서비스는 설비 상태를 NORMAL과 ALARM 두 가지로 관리하고, 상태가 바뀔 때만 메시지를 발행한다(TODO 2).

알람은 NORMAL에서 ALARM으로 바뀔 때 한 번만 발행한다



알람 서비스 상태 전이 · 상태 2개 (TODO 2)

ALARM이 이어지는 동안에는 알람을 다시 보내지 않으므로, 이상 이벤트 하나에 알람은 한 건만 나간다.

토픽과 메시지 형식은 다음 표를 따른다. 모든 토픽은 [USER_ID]/factory/line1/ 아래에 있다.

TODO	토픽	시점	QoS · retain	메시지 예
1	press01/telemetry, press01/sim (구독)	연결 직후	QoS 1	—

TODO	토픽	시점	QoS · retain	메시지 예
3	press01/alarm/[유형]	NORMAL → ALARM	QoS 1 · retain 없음	`{"seq": 1412, "ts": 1790751430.2, "type": "stuck", "score": 12.5}`
4	press01/health	시작 시, 상태가 바뀔 때마다	QoS 1 · retain	`{"state": "ALARM", "type": "stuck", "seq": 1412}`
5	alarm_service/status	연결 직후 ONLINE, 비정상 종료 시 브로커가 OFFLINE	QoS 1 · retain · Will	`{"service": "alarm_service", "status": "ONLINE"}`

5. 실행 방법과 자동 검사

개발 중에는 quick 모드와 모니터로 결과를 바로 확인하고, 제출 전에 full 모드로 자동 검사를 한 번 실행한다. 세 터미널 모두 다섯 파일이 있는 같은 폴더에서 실행하고, 처음에 `pip install paho-mqtt numpy`를 실행한다.

개발 중: quick 모드 (1회 약 1분 30초)

1. 터미널 1: `python watch_alarms.py` (계속 켜 둔다)
2. 터미널 2: `python alarm_service.py` (코드를 고칠 때마다 다시 실행)
3. 터미널 3: `python sensor_simulator.py`

모니터는 센서 값을 출력하지 않고 알람 서비스가 발행한 결과만 보여준다. TODO를 하나 구현할 때마다 해당 줄이 나타나는지 확인하면 된다. TODO 5는 [status], TODO 4는 [health], TODO 1·2·3은 [알람] 줄로 확인한다.

```
[16:42:21] [status] ONLINE
[16:42:24] ===== 시뮬레이터 시작 · quick 모드 · 900개 샘플 · 이상 4건 =====
[16:42:27] [진행] 학습 구간 종료 · 이상 판정 구간 시작 (seq 300)
[16:42:29] [정답] seq 512  stuck 이상 시작
[16:42:29] [알람] seq 525  stuck     OK 정답과 일치 (1.3 s 후)
[16:42:29] [health] ALARM · stuck
[16:42:30] [health] NORMAL
[16:42:34] ===== 종료 · 탐지 4/4 · 오탐 0 · 중복 0 · 유형 불일치 0 =====
```

알람이 오지 않으면 [놓침], 상태 전이가 틀리면 [중복 알람]이 표시된다. 시뮬레이터를 다시 실행하면 모니터는 새 실행을 이어서 표시한다.

제출 전: full 모드 (1회 약 10분)

1. sensor_simulator.py의 RUN_MODE를 "full"로 바꾼다.
2. 터미널 1: `python check_alarms.py`
3. 터미널 2: `python alarm_service.py`
4. 터미널 3: `python sensor_simulator.py`
5. 시뮬레이터가 끝나면 검사기가 [1]~[7] 결과를 출력한다. 이어서 안내에 따라 60초 안에 터미널 2에서 Ctrl + C를 누르면 [8]을 검사하고 check_result.json에 저장한다.

검사 항목은 다음과 같다. quick 모드에서도 검사기를 쓸 수 있지만, 결과 파일에 mode가 quick으로 기록되어 제출용으로 인정되지 않는다.

검사	확인 내용	관련 TODO
[1] 알람 토픽	alarm/[유형] 토픽이고 payload의 type과 일치	3
[2] 알람 형식	seq, ts, type, score 포함	3
[3] 알람 QoS	QoS 1로 발행	3
[4] 이벤트 알람	이상 12건 중 11건 이상에 알람 발행	1, 2
[5] 중복 알람 없음	이벤트를 넘는 알람 2건 이하	2
[6] health 보관	종료 시점의 retain된 health가 NORMAL	4
[7] 서비스 ONLINE	retain된 status가 ONLINE	5
[8] Will 메시지	Ctrl + C 종료 시 OFFLINE 수신	5

제공 모델은 이상 12건을 모두 올바른 유형으로 판정한다. 따라서 TODO를 올바르게 구현하면 8개 항목을 모두 통과한다.

6. 제출물

제출물은 다음 2가지다.

1. 전체 코드: alarm_service.py를 포함한 과제 폴더 전체(zip)
2. 과제 결과물 분석 및 정리 문서(PPT): 구현 내용과 실행 결과를 분석·정리하고, 아래 Q1~Q4에 대한 답을 반드시 포함한다.

문서에 포함해야 하는 질문은 다음과 같다.

- Q1. 관제 화면에서 stuck 알람만 받으려면, 그리고 모든 유형의 알람을 받으려면 각각 어떤 토픽 필터를 구독해야 하는가?
- Q2. health를 retain 없이 발행하면, 이상 발생 후 늦게 켜진 관제 화면에서 어떤 문제가 생기는가?

- Q3. 알람에 QoS 0 대신 QoS 1을 사용하는 이유는 무엇인가?
- Q4. 상태 전이(TODO 2) 없이 이상 판정마다 알람을 보내면 현장 운영에서 어떤 문제가 생기는가?

7. 유의사항

- anomaly_model.py, sensor_simulator.py, check_alarms.py는 수정하지 않는다. 단, USER_ID는 수정한다.
- 알람 서비스는 시뮬레이터보다 먼저 실행한다. 늦게 실행하면 학습 구간을 받지 못해 판정을 시작하지 않는다.
- [8] 검사에서는 Ctrl + C로 종료한다. 종료 처리에서 disconnect()를 호출하면 Will 메시지가 폐기되어 통과하지 못한다.
- 공개 브로커에 접속할 수 없으면 실습 FAQ를 참고하여 모바일 핫스팟으로 전환한다. 로컬 브로커를 쓸 때는 환경 변수 MQTT_BROKER로 주소를 지정한다.
- 과제를 마친 뒤에는 health와 status 토픽에 빈 페이로드를 retain으로 발행하여 보관 메시지를 정리한다.